Module 2 — Collections Framework

> Highest priority and the single most-asked coding-adjacent topic. “How does HashMap work internally?” is asked in *almost every* Java interview at TCS/Infosys/Cognizant and product companies alike. Know the internals cold.

Node.js bridge: JS has `Array`, `Map`, `Set`, `Object`. Java gives you a rich typed hierarchy with explicit complexity guarantees and thread-safe variants. `HashMap` ≈ JS `Map` but with load factor, buckets, and treeification you can be quizzed on.

2.1 Collection Hierarchy

Core Concept

```
                Iterable
                   |
               Collection
     +-------------+-------------+
    List          Set          Queue
     |             |             |
ArrayList     HashSet        LinkedList (also List)
LinkedList    LinkedHashSet  PriorityQueue
Vector        TreeSet(NavigableSet)  ArrayDeque (also Deque)
(Stack)

                   Map   (NOT a Collection)
     +-------------+-------------+
  HashMap     LinkedHashMap   TreeMap(NavigableMap)
  Hashtable   ConcurrentHashMap
```

- **`Map` is not a `Collection`** — it’s a separate root (key→value). Common trap.
- **List** = ordered, indexed, duplicates allowed.
- **Set** = no duplicates.
- **Queue/Deque** = FIFO / double-ended.

Interfaces vs implementations (senior framing)

Program to the interface: `List<X> l = new ArrayList<>();`, `Map<K,V> m = new HashMap<>();`. Choose the implementation for the access pattern and thread model.

Cheat Sheet — pick the right one

Need	Use
Indexed, mostly reads	`ArrayList`
Frequent add/remove at ends, queue/deque	`ArrayDeque` (not `LinkedList`)
Unique, fast lookup	`HashSet`
Unique + insertion order	`LinkedHashSet`
Unique + sorted	`TreeSet`
Key→value fast	`HashMap`
Key→value + order	`LinkedHashMap` (also LRU)
Key→value + sorted	`TreeMap`
Concurrent map	`ConcurrentHashMap`
Priority ordering	`PriorityQueue`

2.2 ArrayList — Dynamic Array, Resize, Complexity

1. Why Interviewers Ask This

It's the default `List` and the base for "what's the complexity of `add`" and "how does it grow" questions.

2. Core Concept

Resizable array backed by `Object[] elementData`. Random access by index is O(1).

3. Internal Working — growth

- Default capacity **10** (lazily allocated on first add).
- When full, it grows by **~50%**: `newCap = oldCap + (oldCap >> 1)`. (`ArrayList`; `Vector` doubles.)
- Growth = allocate new array + `System.arraycopy` old → new. That single add is O(n), but **amortized O(1)** across many adds.
- `add(index, e)` and `remove(index)` shift elements → O(n).
- Not thread-safe.

4. Memory Diagram

```
size=3, capacity=4:  elementData -> [ A | B | C | _ ]
add(D): full -> newcap = 4 + 2 = 6, copy:
                    [ A | B | C | D | _ | _ ]
```

5. Real Production Example

Loading 1M rows from DB into `new ArrayList<>()` triggers ~20 resizes/copies. Pre-size with `new ArrayList<>(1_000_000)` to avoid the copy churn — a real latency/GC win.

6. Most Asked Questions

- How does ArrayList grow? *(1.5x, arraycopy)*
- Complexity of get/add/add-at-index/remove? *(O(1)/amortized O(1)/O(n)/O(n))*
- ArrayList vs Array? *(dynamic vs fixed, generics, autobox overhead)*
- ArrayList vs LinkedList? (see below)
- Is ArrayList thread-safe? How to make it? *(`Collections.synchronizedList` / `CopyOnWriteArrayList`)*

7. Traps

- Saying it doubles (that's `Vector`; ArrayList is 1.5x).
- Removing while iterating with a for-each → `ConcurrentModificationException` (use `Iterator.remove()`).

8. Best Answer

> "`ArrayList` is a dynamic array; get is O(1). When capacity is exceeded it grows by 50% and arraycopies, so add is amortized O(1) but inserts/removes in the middle are O(n) due to shifting. I pre-size it when I know the count to avoid resize churn."

9. Coding Example

```
List<String> list = new ArrayList<>(1000);       // pre-sized
list.add("a");
// Safe removal during iteration:
Iterator<String> it = list.iterator();
while (it.hasNext()) { if (it.next().isEmpty()) it.remove(); }
```

10. Follow-ups

- Remove all even numbers safely (Iterator vs `removeIf`).

- Convert array↔list (`Arrays.asList` gotcha: fixed-size, backed by array).

11 & 12. Summary + Cheat

Dynamic array, 1.5x growth, O(1) get, O(n) middle ops. Pre-size when possible.

2.3 LinkedList

Doubly-linked list; implements `List` **and** `Deque`. - get(index) = O(n) (walks nodes); add/remove at ends = O(1); add/remove in middle = O(1) *if you hold the node* but O(n) to find it. - Higher memory per element (two pointers + object header). - **In practice, prefer** `ArrayList` for lists and `ArrayDeque` for queue/stack — `LinkedList` is rarely the right answer despite the textbook "fast inserts".

Best answer trap: "Use LinkedList for frequent insertions" — only true when you already have the node reference; otherwise finding the position is O(n). CPU cache locality makes `ArrayList` win in most real workloads.

2.4 HashMap — the flagship internals question

1. Why Interviewers Ask This

The most-asked Java question, period. It tests hashing, buckets, collisions, equals/hashCode, resize, and (post-Java 8) treeification.

2. Core Concept

Stores key→value in an **array of buckets**. The key's `hashCode()` decides the bucket; `equals()` resolves within a bucket. Average O(1) get/put.

3. Internal Working (Java 8+)

1. `hash(key)` = `h = key.hashCode(); h ^ (h >>> 16)` — spreads high bits into low bits so poorly-distributed hashcodes still scatter.
2. **Bucket index** = `(n - 1) & hash` where `n` = table length (always a power of 2, so `&` = fast modulo).
3. Each bucket is a `Node` **linked list**; on collision, append (Java 8 appends at tail).
4. **Put**: if key's hash+`equals` matches an existing node → replace value; else add node.
5. **Load factor** default **0.75**. When `size > capacity * loadFactor`, **resize**: capacity doubles (16→32→…), and every node is **rehashed** into the new table.
6. **Treeification**: if a single bucket's list length ≥ **8** *and* table capacity ≥ **64**, the list converts to a **red-black tree** → worst case O(log n) instead of O(n). If it shrinks below **6**, it **untreeifies** back to a list.
7. Default initial capacity **16**. Null key allowed (stored in bucket 0); null values allowed.

4. Memory Diagram

```
capacity=16, loadFactor=0.75 -> resize at size 12
table:
 [0] -> null
 [1] -> (k1,v1) -> (k9,v9)          <- collision: linked list in bucket 1
 [2] -> null
 ...
 [5] -> (k5,v5) -> ... (>=8 & cap>=64) -> converts to Red-Black TREE

index = (n-1) & hash(key)      hash(key)=h ^ (h>>>16)
```

5. Real Production Example

Using a mutable object as a key (its `hashCode` changes after insertion) makes entries "disappear" — you can't `get` them because the bucket index changed. Always use **immutable keys** (`String`, `Long`, records). Overriding `equals` but not `hashCode` breaks map lookups — a classic prod bug.

6. Most Asked Interview Questions

- Explain HashMap internal working. *(hash → bucket → list/tree → equals)*
- What is load factor? Why 0.75? *(space/time tradeoff; higher = more collisions, lower = wasted space)*
- How/when does resize happen? *(size > cap*0.75 → double + rehash)**
- What is treeification and its thresholds? *(list→tree at 8 & cap≥64)*
- Why must capacity be a power of 2? *(`(n-1) & hash` works as fast modulo)*
- equals/hashCode contract? *(equal → same hash; unequal may collide)*
- What happens with a null key? *(bucket 0)*
- Is HashMap thread-safe? What breaks under concurrency? *(no; resize can corrupt/infinite-loop in Java 7; use CHM)*
- **Follow-up:** difference between Java 7 and 8 HashMap? *(Java 8 added treeification, tail-insert to avoid the Java 7 resize infinite-loop)*

7. Interview Traps

- Overriding `equals` without `hashCode` (or vice versa).
- Saying collisions become a tree immediately (needs length ≥8 **and** capacity ≥64).
- Saying HashMap maintains order (it doesn't — use `LinkedHashMap`).
- Claiming HashMap put/get is always O(1) (worst case O(log n) after treeify, O(n) before Java 8).

8. Best Answer

> "`HashMap` is an array of buckets. I compute a spread hash (`h ^ h>>>16`) and index with `(n-1)&hash`. Collisions chain in a linked list; from Java 8, once a bucket has ≥8 nodes and the table is ≥64, it becomes a red-black tree for O(log n) worst case. It resizes — doubling capacity and rehashing — when size exceeds capacity × 0.75. Keys must be immutable with a consistent `equals`/`hashCode`, or lookups break."

9. Coding Example — correct equals/hashCode

```
public final class UserId {
    private final String tenant;
    private final long id;
    public UserId(String tenant, long id){ this.tenant = tenant; this.id = id; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof UserId u)) return false;
        return id == u.id && tenant.equals(u.tenant);
    }
    @Override public int hashCode() { return Objects.hash(tenant, id); }
}
// Now safe as a HashMap key.
```

10. Follow-up Coding Questions

- Find the first non-repeating char using a `LinkedHashMap<Character,Integer>`.
- Group anagrams using `HashMap<String,List<String>>`.
- Implement a simple hash map from scratch (array + linked list + resize).

11. Summary

Array of buckets, spread hash, `(n-1)&hash` index, chaining → tree at 8/64, resize at 0.75. Immutable keys + correct equals/hashCode.

12. Cheat Sheet

```
default cap 16, LF 0.75, resize=double+rehash
index=(n-1)&hash ; hash=h^(h>>>16)
treeify: bucket>=8 AND cap>=64 ; untreeify<6
null key -> bucket 0 ; not thread-safe
equals true => hashCode equal (mandatory)
```

2.5 ConcurrentHashMap (CHM)

Core Concept & Internal Working

Thread-safe `Map` without locking the whole table. - **Java 7:** segment locking (default 16 segments) — lock striping. - **Java 8+:** dropped segments. Uses the **bucket array + CAS** for empty-bucket inserts and **`synchronized` on the bucket's head node** for collisions. Reads are lock-free (`volatile` nodes). Also treeifies like HashMap. - **No null keys or values** (ambiguity between "absent" and "null" in concurrent `get`). - `size()` is approximate under concurrency (uses `baseCount` + counter cells). - Atomic composites: `putIfAbsent`, `compute`, `computeIfAbsent`, `merge` — use these instead of check-then-act.

Memory Diagram

```
Java 8 CHM:
 bucket empty  -> CAS in new node (no lock)
 bucket has head-> synchronized(head){ append / update }   (fine-grained)
 reads          -> volatile, lock-free
```

Interview Q&A

- **HashMap vs Hashtable vs CHM?** HashMap not thread-safe; Hashtable synchronizes every method (whole-object lock, legacy, slow); CHM locks per-bucket → high concurrency.
- **Why no null in CHM?** Can't distinguish "key absent" from "mapped to null" without locking.
- **Why not just `Collections.synchronizedMap`?** That wraps a single lock — no concurrency; CHM allows concurrent reads and striped writes.
- **`computeIfAbsent` use?** Atomic lazy init of cache entries (avoids race of `if(!contains) put`).

Best Answer

> "CHM gives thread-safety with high throughput: in Java 8 it CASes into empty buckets and only `synchronized`-locks the individual bucket head on collision, while reads stay lock-free via volatile. It forbids nulls and offers atomic `computeIfAbsent`/`merge` so I don't do racy check-then-act. `Hashtable` locks the whole map — I never use it."

2.6 Sets — HashSet, LinkedHashSet, TreeSet

	HashSet	LinkedHashSet	TreeSet
Backing	HashMap	LinkedHashMap	TreeMap (red-black)
Order	none	insertion	sorted
Null	1 null	1 null	no null (NPE on compare)
get/add/contains	O(1)	O(1)	O(log n)

- **`HashSet` is literally a `HashMap`** with a dummy `PRESENT` value for every key. Same equals/hashCode rules apply.

- **`TreeSet`** needs elements `Comparable` or a `Comparator`; keeps sorted order; offers `first/last/ceiling/floor/headSet/tailSet`.
- **`LinkedHashSet`** = predictable iteration order (insertion), slightly more memory.

2.7 Queues & Deque — PriorityQueue, ArrayDeque

PriorityQueue

- A **binary min-heap** (array-backed). `peek` = min = O(1); `offer`/`poll` = O(log n).
- Ordering by natural order or a `Comparator`. **Not** sorted iteration — only the head is guaranteed smallest.
- Not thread-safe (use `PriorityBlockingQueue`).
- Classic uses: Dijkstra, top-K, task scheduling by priority.

ArrayDeque

- Resizable-array double-ended queue. **Preferred stack and queue** (faster than `Stack` and `LinkedList`; no synchronization overhead).
- `offerFirst/offerLast/pollFirst/pollLast`, or `push/pop` for stack semantics. No null allowed.

Memory Diagram — PriorityQueue heap

```
Array: [1, 3, 2, 7, 4]  represents min-heap:
             1
           /   \
          3     2
         / \
        7   4
parent(i)=(i-1)/2 ; children=2i+1, 2i+2 ; poll() removes root, sifts down
```

2.8 Maps — TreeMap, LinkedHashMap

TreeMap

- **Red-black tree**, sorted by key (natural or `Comparator`). O(log n) ops.
- `NavigableMap`: `firstKey/lastKey/ceilingKey/floorKey/headMap/tailMap/subMap`.
- No null keys. Use when you need **range queries** or sorted iteration.

LinkedHashMap

- HashMap + doubly-linked list across entries → predictable **insertion** (or **access**) order.
- **`accessOrder=true` + override `removeEldestEntry`** → a ready-made **LRU cache** (top interview coding ask).

```
class LRUCache<K,V> extends LinkedHashMap<K,V> {
    private final int cap;
    LRUCache(int cap){ super(cap, 0.75f, true); this.cap = cap; } // access-order
    @Override protected boolean removeEldestEntry(Map.Entry<K,V> e){ return size() > cap; }
}
```

2.9 Comparable vs Comparator

	Comparable	Comparator
Method	`compareTo(o)`	`compare(a,b)`
Where	inside the class	external/separate
Count	one “natural” order	many orders
Example	`String`, `Integer`	sort users by age then name

```
class Emp { String name; int age; }
List<Emp> list = ...;
list.sort(Comparator.comparingInt((Emp e) -> e.age)   // Comparator, Java 8 style
        .thenComparing(e -> e.name));
// reverse: .reversed()
```

- `compareTo/compare` return negative/zero/positive.
- **Trap:** `compareTo` inconsistent with `equals` breaks `TreeSet`/`TreeMap` (they use compare, not equals — two “equal by compare” elements are deduped!).

2.10 Iterator · Fail-Fast vs Fail-Safe · ConcurrentModificationException

Core Concept

- **Iterator** = cursor to traverse a collection; supports safe `remove()`.
- **Fail-fast** iterators (ArrayList, HashMap) track a **`modCount`**. If the collection is structurally modified during iteration (except via the iterator), the next `next()` throws **`ConcurrentModificationException`** (CME) — best-effort, not guaranteed.
- **Fail-safe** iterators (`CopyOnWriteArrayList`, `ConcurrentHashMap`) iterate over a **snapshot** or tolerate concurrent modification → no CME, but may not reflect latest writes.

Internal Working — modCount

```
expectedModCount = modCount (at iterator creation)
list.add(x) during loop -> modCount++
it.next() -> if (modCount != expectedModCount) throw ConcurrentModificationException
```

> CME can happen in a **single thread** too — e.g., adding/removing inside a for-each loop.

Best Answer & Fixes

> “Fail-fast iterators throw `ConcurrentModificationException` when they detect structural change via `modCount`, even single-threaded — like removing inside a for-each. I fix it with `Iterator.remove()`, `removeIf()`, collecting to a new list, or a concurrent/copy-on-write collection when I truly have multiple threads.”

```
// WRONG - throws CME:
for (String s : list) if (s.isBlank()) list.remove(s);
// RIGHT:
list.removeIf(String::isBlank);
```

2.11 Collections & Arrays Utility Classes

`Collections` (operates on Collection objects)

`sort`, `reverse`, `shuffle`, `binarySearch`, `max/min`, `frequency`, `unmodifiableList`, `synchronizedList/Map`, `emptyList`, `singletonList`.

`Arrays` (operates on arrays)

`sort`, `binarySearch`, `fill`, `copyOf`, `equals`, `asList`, `stream`, `toString`, `parallelSort`. - **Trap:** `Arrays.asList(arr)` returns a **fixed-size** list backed by the array — `add`/`remove` throw `UnsupportedOperationException`; changes write through to the array. For a mutable copy: `new ArrayList<>(Arrays.asList(arr))`. - `List.of(...)` / `Map.of(...)` (Java 9) → **immutable**; `add` throws.

2.12 Time & Space Complexity (must-memorize table)

Structure	get/contains	add	remove	Notes
ArrayList	O(1) index / O(n) search	amortized O(1) end, O(n) middle	O(n)	array copy on resize
LinkedList	O(n)	O(1) ends	O(1) w/ node	2 pointers/node
ArrayDeque	O(n) search	O(1) ends	O(1) ends	best stack/queue
HashMap/HashSet	O(1) avg, O(log n) treeified	O(1) avg	O(1) avg	0.75 LF
LinkedHashMap/Set	O(1)	O(1)	O(1)	keeps order
TreeMap/TreeSet	O(log n)	O(log n)	O(log n)	sorted, red-black
PriorityQueue	O(1) peek, O(n) contains	O(log n)	O(log n)	binary heap
ConcurrentHashMap	O(1) avg	O(1) avg	O(1) avg	lock-striped/CAS

Module 2 — Top 25 Interview Questions (senior answers)

1. **Is `Map` a `Collection`?** No — separate hierarchy.
2. **How does HashMap work internally?** hash → `(n-1)&hash` bucket → list/tree → equals.
3. **Load factor & default capacity?** 0.75, 16; resize doubles at size>cap*0.75.
4. **Treeification thresholds?** ≥8 in a bucket **and** cap≥64 → red-black tree; <6 untreeify.
5. **Why capacity power of 2?** `(n-1)&hash` acts as fast modulo and distributes evenly.
6. **equals/hashCode contract?** Equal objects must have equal hashCode; needed for map/set correctness.
7. **What if you override equals but not hashCode?** Lookups fail — objects land in wrong/duplicate buckets.
8. **HashMap vs Hashtable vs ConcurrentHashMap?** Unsynchronized vs whole-object-locked vs bucket-level/CAS.
9. **Why no null in ConcurrentHashMap?** Can't distinguish absent vs null concurrently.
10. **ArrayList vs LinkedList?** Array (O(1) get, cache-friendly) vs nodes (O(1) ends only).
11. **How does ArrayList grow?** 1.5x + arraycopy; amortized O(1) add.
12. **Vector vs ArrayList?** Vector synchronized + doubles; ArrayList not synced + 1.5x.
13. **HashSet vs TreeSet vs LinkedHashSet?** Unordered O(1) / sorted O(log n) / insertion-order.
14. **Comparable vs Comparator?** Natural single order in-class vs multiple external orders.
15. **compareTo inconsistent with equals — impact?** TreeSet/TreeMap drop "equal" elements.
16. **Fail-fast vs fail-safe?** modCount CME vs snapshot iteration.
17. **When does ConcurrentModificationException happen?** Structural change during iteration (even single-thread).
18. **How to remove during iteration?** `Iterator.remove()` / `removeIf`.

19. **PriorityQueue internals?** Binary min-heap; O(log n) offer/poll.
20. **Best stack/queue implementation?** `ArrayDeque` (not `Stack`/`LinkedList`).
21. **How to build an LRU cache?** `LinkedHashMap` access-order + `removeEldestEntry`.
22. **`Arrays.asList` gotcha?** Fixed-size, array-backed; wrap in `new ArrayList<>()`.
23. **Immutable collections?** `List.of`, `Map.of`, `Collections.unmodifiableList`.
24. **When TreeMap over HashMap?** Sorted keys / range queries (`ceilingKey`, `subMap`).
25. **Complexity of HashMap get worst case?** O(log n) after treeify (O(n) pre-Java 8).

Module 2 — Top Coding Questions

- Implement LRU cache (LinkedHashMap and manual HashMap+DLL).
- First non-repeating character (LinkedHashMap).
- Group anagrams; find duplicates; count word frequency.
- Top-K frequent elements (HashMap + PriorityQueue).
- Merge/sort by multiple fields with `Comparator.comparing().thenComparing()`.
- Detect and remove duplicates preserving order (LinkedHashSet).
- Implement your own HashMap (buckets + resize).

Module 2 — Common Follow-ups

- “Java 7 vs Java 8 HashMap?” (treeify + tail-insert fixed resize infinite loop.)
- “Why did old HashMap infinite-loop under concurrency?” (Java 7 head-insert reversed list on concurrent resize.)
- “How is CHM `size()` computed?” (baseCount + CounterCells; approximate.)
- “What happens to iteration order after resize?” (rehash changes bucket order; never rely on HashMap order.)

Module 2 — One-Page Cheat Sheet

```
Map != Collection. List(dup,ordered) Set(unique) Queue(FIFO) Deque(both)
ArrayList: dynamic array, 1.5x grow, O(1) get, O(n) middle
LinkedList: doubly-linked, O(1) ends; prefer ArrayList/ArrayDeque
HashMap: cap16 LF0.75, index=(n-1)&hash, hash=h^h>>>16, chain->tree(8 & cap>=64), resize=double+rehash
  immutable keys + equals/hashCode required; null key->bucket0
CHM: Java8 CAS empty + synchronized head; no nulls; computeIfAbsent atomic
Sets: HashSet=HashMap; LinkedHashSet=order; TreeSet=sorted O(log n)
Maps: TreeMap sorted(rbtree); LinkedHashMap=order/LRU(accessOrder+removeEldestEntry)
PriorityQueue=min-heap O(log n); ArrayDeque=best stack/queue
Comparable=natural(compareTo); Comparator=custom(compare, thenComparing, reversed)
fail-fast(modCount->CME) vs fail-safe(snapshot). remove via Iterator.remove/removeIf
Arrays.asList = fixed-size backed array; List.of = immutable
```

Module 2 — Mock Interview (answer, then continue)

1. “Walk me through `map.put(key, value)` end to end, including collision and resize.”
2. “You override `equals()` on an entity but forget `hashCode()`. What breaks and why?”
3. “How would you build an LRU cache in Java? Now make it thread-safe.”
4. “Explain how ConcurrentHashMap achieves thread-safety without locking the whole map.”
5. “Why is `ArrayDeque` preferred over `Stack` and `LinkedList`?”
6. “You get a `ConcurrentModificationException` while filtering a list in one thread. Why, and 3 ways to fix it?”

Model answers are in the sections above. Continue to Module 3 when ready.